

Lab0 实验报告

一、文档中要求回答的问题

1. 你之前有多人协同开发的经历吗？如果有，你们使用什么方式分工协作？

有过非常初步的一次开发尝试，完全是一人在负责开发编写，另外二人将想法与操作全部通过 word 文档发送给负责代码的人，现在再回溯会发现效率很低下。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

1. 可以只把比较小的改动放进暂存区，分多次提交，让每次提交都对应一个单独的逻辑改进，历史记录更清晰。

2. 提交前可以先预览暂存区里的内容，确认无误再提交，还能把不想提交的文件或代码片段留在工作区，提供了反悔和检查的机会。

3. 工作区域可能很杂乱，有临时文件和调试代码，但暂存区只放需要提交的最终内容，保证提交更整洁。

3. `git branch`` 和 ``git branch -a`` 的区别是什么？

查阅资料可知，二者区别核心在于是否显示远程仓库的分支：`git branch` 只显示本地分支，即当前电脑上创建的分支，看不到远程仓库（如 GitHub/GitLab）上有什么分支；`git branch -a` 则显示所有分支，`-a` 是 `--all` 的缩写。

二、使用模板仓库建立个人仓库，完成 `main.c` 文件中的 TODO 部分并进行一次 `commit`。

实验步骤：

1. 阅读课程界面上的资料，创建个人仓库。
2. 将仓库克隆到本地。
3. 修改 `main.c` 文件，填入代码并保存。
4. 在终端执行 `git add` 等命令（中间网络卡顿导致反复交了很多次）。
5. 执行 `git push` 推送到 GitHub 远程仓库。

以下分别为 Visual Studio Code 和仓库的截图：

```
C main.c M X
C main.c > main()
1 #include <stdio.h>
2 int main() {
3     printf("I am very happy to learn data structures.\n");
4     return 0;
5 }
```

Commits

main	All users	All time
Commits on Sep 26, 2026		
一些原因导致的修复 vs-code-user-igtm committed 12 minutes ago	1a359d8	<>
Merge branch 'main' of https://github.com/vs-code-user-igtm/TestLab vs-code-user-igtm committed 16 minutes ago	3d60b15	<>
Add files via upload vs-code-user-igtm authored 19 minutes ago	Verified fe0ceb6	<>
实验二 TODO 部分 vs-code-user-igtm committed 42 minutes ago	04d9496	<>
实验二 TODO 部分 vs-code-user-igtm committed 1 hour ago	1a02732	<>

三、拓展阅读与思考

阅读内容概括：

（1）《Commit Message 规范》

这篇文章主要讲的是提交代码时，附带的说明应该怎么写才规范。它推荐了一种叫 Angular 的格式，核心是“类型：简短描述”，比如新增功能用 feat，修 bug 用 fix。文章还强调，写清楚提交说明不仅方便自己以后回头看，也能让队友快速明白你改了什么，同时还能用工具自动生成更新日志。

（2）《Git Flow 分支控制》

这篇文章介绍了一套比较经典的分支管理方法，也就是 Git Flow。它的思路是把代码分成几条不同的“线路”：主分支放稳定版本，开发分支放最新的开发成果，每个新功能单独开一条分支，做完再合并回去。如果线上出了紧急问题，还有专门的热修复分支来处理。这套方法的好处是让开发、发布和修 bug 互不干扰，适合有明确发布周期、多人协作的正规项目。

（3）《语义化版本》

这篇文章讲的是软件的版本号应该怎么定。它建议用“主版本号. 次版本号. 修订号”这三个数字来表示，比如 1.2.3。改动很大、和老版本不兼容时，第一个数字加一；加了新功能但兼容旧版时，第二个数字加一；只是修了小问题时，第三个数字加一。这样一来，别

人光看版本号的变化，就能大致判断这次更新会不会影响自己，从而减少升级时踩坑的风险。

通过阅读这三篇文章，我认为学习 Git 首先是为了解决“改错了怎么办”这个比较实际的问题。以其他方式写代码时，改完不满意想退回去，只能靠手动备份，经常越弄越乱。有了 Git，每次改动都可以存一个记录，随时能翻回任何一个版本，更加可靠。

其次，它让多人合作写代码变得可行。大家一起做一个项目时，如果都在同一个文件上改，很容易互相覆盖。Git 的分支功能可以让每个人先在自己的分支上干活，做完再合并，冲突也能被清楚地标出来，方便解决。

最后，Git 已经是现在开发工作的基本工具，通用性很强。

四、Git 分支管理学习

实验步骤

1. 新建 feature 分支，修改并提交
2. 切回 main 分支，修改并提交
3. git merge feature 制造冲突。
4. 通过选择“保留双方更改”解决冲突，提交最终版

以下是三版不同代码和冲突的截图：

```
1  #include <stdio.h>
2  int main() {
3      printf("This is the MAIN branch version.\n");
4      return 0;
5  }
```

```
1  #include <stdio.h>
2  int main() {
3      printf("This is the FEATURE branch version\n");
4      return 0;
5  }
```

```

1  #include <stdio.h>
2  int main() {
    采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
3  <<<<<< HEAD (当前更改)
4      printf("This is the MAIN branch version.\n");
5  =====
6      printf("This is the FEATURE branch version\n");
7  >>>>>> feature (传入的更改)
8      return 0;
9  }

```

```

1  #include <stdio.h>
2  int main() {
3      printf("This is the MAIN branch version.\n");
4      printf("This is the FEATURE branch version\n");
5      return 0;
6  }

```

五. 我的建议

这次的 lab 主要是帮助同学们入门，对于学生来讲其实是十分重要的。作为新手，我有两点建议。首先，建议在最前面的说明中增加本地与远程仓库关系的图示说明，因为刚接触 Git 时，很容易混淆工作区、暂存区、终端、本地仓库、远程仓库这几个概念，常常不知道某一段工作应该在哪里完成。如果文档里能加一张流程图，标出 git add、git commit、git push 分别发生在哪一步以及哪里，会更容易理解。其次，建议对网络问题给出替代方案，因为国内直连 GitHub 经常失败，比如我在某一次提交中卡了几个小时都没交上去，建议文档里说明可以使用加速工具或其他辅助方式，避免因网络问题卡在最后一步推送。